Agentic AI Course Roadmap

Oct 9, 2026 · @Andrew Seaman

How to use this map

Use this doc as a home base: zoom out to place a new idea, zoom in to try it, then zoom back out to file it.

Bottom-up alone buries you because every detail (an import error, a pip flag) feels equally urgent. Top-down alone feels shaky because the big picture seems unearned. You don't have to choose: the big picture is a set of shelves, and the details are what you put on them.

1. **Zoom out (2 minutes, before a lesson).** Find the lesson in the Unit 1 table and read its one idea. Then find the part it adds on the agent diagram.
2. **Zoom in (the lesson itself).** Run the code. When a detail stalls you, ask: does this change the one idea? If not, park it on a "later" list and keep going.
3. **Zoom back out (2 minutes, after).** Write one sentence: "Today my agent gained ___, because ___."

Hang every new term on three questions:

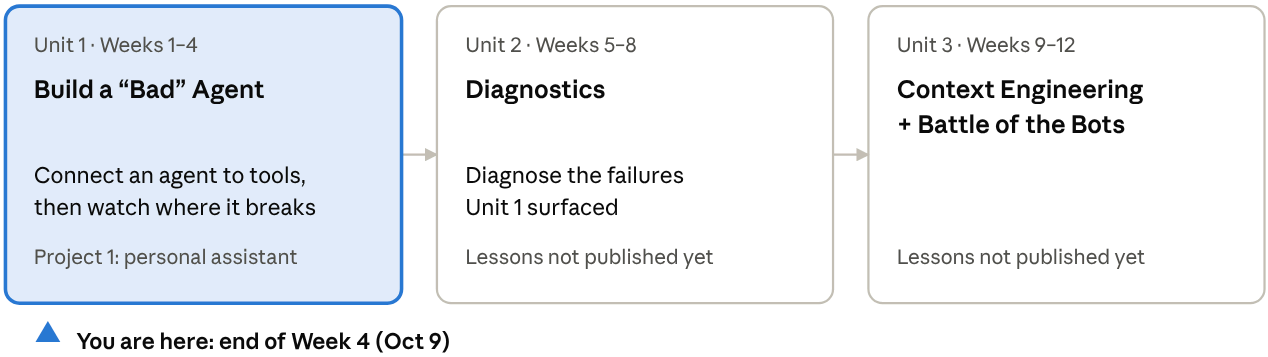
- What part of the agent does it touch?
- What problem made it necessary?
- Who owns and runs it?

The course is built this way too. Most lessons open with an agent failing (Ludwig's support bot, Mateo's "I can't") and ask what the agent would need. That question is your top-down view, and each lesson lets you check it.

The whole course in one picture

The course runs 12 weeks in three 4-week units, and each feeds the next: Unit 1's failures become Unit 2's raw material.

Unit 1 builds an agent; its failures drive Unit 2

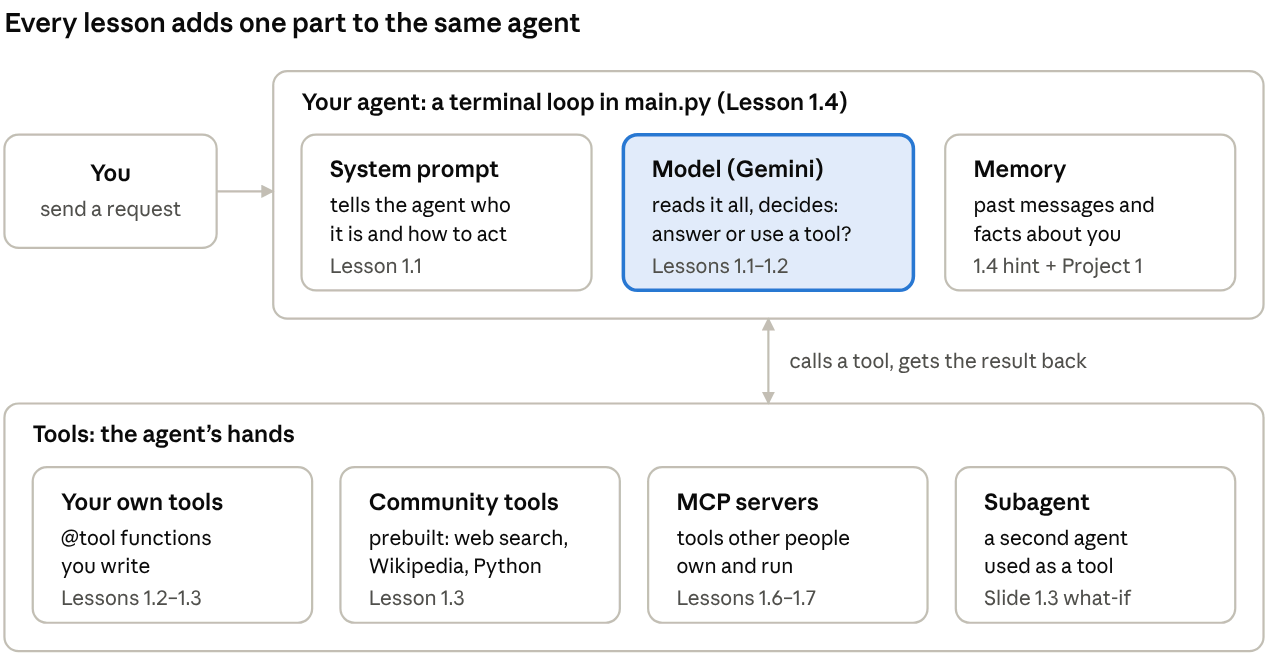


course structure · 3 units, 12 weeks

Unit 1's agent is "bad" on purpose: you build something powerful but hard to control, then watch it fail. Only Unit 1 is published so far. The course objectives show where it ends up: build agent harnesses, evaluate agents with observability tools, turn vague goals into measurable criteria, and form a personal ethic for using agents.

What's inside an agent

An agent is a model that reads its system prompt, the conversation so far, and its tools' descriptions, then decides whether to answer or call a tool.



agent anatomy · 3 core parts, 4 kinds of tools

Lesson 1.2 walks the loop: you ask "What is 99 times 47?", the model reads its tool list, picks the calculator because its docstring says to, gets back 4653, and answers. Every Unit 1 lesson adds or improves one box here, which is why the lesson numbers sit on the boxes.

Unit 1, step by step

Unit 1 grows one agent, one capability per lesson: from a demo in Colab to a personal assistant on your own machine, wired to tools you write, tools others prebuilt, and tools others run.

Lesson	Date	What it adds to your agent	The one idea to keep
0 · Get Oriented	Sep 14	Nothing yet: course kickoff	The page is empty on the site.
1.1 · Your First LangChain Agent	Sep 16	A working agent in Colab: a Gemini model, one tool, a system prompt	Your code reaches the model through an API key, and the agent chose to call a tool on its own.

Lesson	Date	What it adds to your agent	The one idea to keep
1.2 · Tools: Giving Your Agent Hands	Sep 18	Your own tools, made with `@tool`	The agent never sees your code. It picks a tool by reading the docstring.
1.3 · Group Day and Community Tools & Real Data	Sep 21	Web search and a local-file tool on the same agent	With several tools, each description must say when to use it and when not to.
1.4 · Starting From Scratch	Sep 23	The agent moves to your computer: VS Code, uv, `.env`, a terminal loop in `main.py`, a public GitHub repo	This repo becomes Project 1. Keys live in `.env`, which git ignores.
1.5 · Review	Sep 25	No new code: review, demos, Project 1 requirements	Intelligence gets more capable once it is organized; next, your agent plugs into other people's systems.
1.6 · Tools You Don't Own	Sep 28	The LangChain docs MCP server; calls become async (`ainvoke`)	Let the system's owner ship the tool. Your agent fetches the current tool list every time it connects.
1.7 · Someone Already Built It	Sep 30	Several ready-made MCP servers (arXiv, filesystem) in one config	stdio: your agent starts the server on your machine. HTTP: the server is already running somewhere else.
1.8 · Ethics discussion and work day	After Sep 30	Build time for Project 1	Not published yet; the outline comes from the end of the 1.7 slides.

The lessons from 1.4 on all edit the same `main.py`, so each one assumes the last one works. If an earlier step is broken, fix it before moving forward.

Project 1: where each requirement is taught

Project 1 is the agent you start in Lesson 1.4 and keep adding to, so the lessons are its build manual. The site lists the due date as TBD, "mid to late week 4 most likely"; counting Sep 14 as Week 1, that is this week, so confirm it in Canvas or Slack today.

Requirement	Where the course teaches it
Public GitHub repo, memory folder in `.gitignore`, fake `example_memory/` committed	Lesson 1.4 (repo, `.gitignore`); the Project 1 page
A main agent that chooses tools and delegates to subagents	Lessons 1.1 and 1.2
Read and write the file system	Lesson 1.3 (local-file tool); Lesson 1.7 (filesystem MCP server)
Conversation history	Lesson 1.4 hint (a thread ID for a checkpointer); the 1.6 slides have you ask your docs-connected agent how to add short-term memory; "Statefulness" on the Unit 1 page, not published yet
Persistent personal memory: classes, goals, resume, preferences	No published lesson yet. The Project 1 page calls it a memory directory, so your file-system tools (1.3, 1.7) are the building blocks
Connect to LangChain MCP	Lesson 1.6 slides: connecting the LangChain docs server is exactly this requirement
At least one subagent (job hunt, money, coursework, study, roommates, your code, translator, learning, or your own idea)	Slide 1.3 "What-If" (a second agent used as a tool); "Subagents" on the Unit 1 page, not published yet
README: what it does, how to run it, its subagents, what it can't do	The Project 1 page
Canvas self-reflection: what works, what doesn't, where it surprised you, with examples	The Project 1 page

Course rules: the Unit 1 page says AI assistance isn't permitted this unit (debugging help comes from class and Slack), and your prompts must be your own. The Project 1 page adds: no coding agents unless you build one with LangChain, ask peers before AI, and ask your own agent before chat tools. This map only summarizes the course pages; the code, prompts, and debugging stay yours. The course also says failures are not a grading penalty, and bringing the project to class unfinished is fine.

Get started: your next steps

Confirm the due date, get a bare agent running on your Mac, then add one requirement at a time in the order the lessons taught them.

1. **Confirm the Project 1 due date** in Canvas or the class Slack channel.
2. **Read three pages for the big picture (about 30 minutes):** the Unit 1 overview, the Project 1 instructions, and the Lesson 1.6 slides.
3. **Set up your Mac** (Prep 1.4 and Lesson 1.7): a GitHub account, VS Code signed in to GitHub, uv, Node.js, and a Gemini API key from Google AI Studio.
   - Check the installs with `uv --version` and `node --version`.
   - Where the slides show `C:/path/to/a/folder`, a Mac path looks like `/Users/<you>/...`.
4. **Build the skeleton** (Lesson 1.4): `uv init`, add `.env` to `.gitignore` before creating `.env`, then a `main.py` loop that echoes your input.
   - Run `git status` before every commit and make sure `.env` is not listed.
5. **Add one capability at a time,** testing after each. This order is my suggestion, not the course's:
   a. One custom `@tool` (Lesson 1.2), to prove the loop works.
   b. The LangChain docs MCP server (Lesson 1.6). From here on, ask your own agent LangChain questions first, which Project 1 requires anyway.
   c. Conversation history (the thread ID hint in Lesson 1.4).
   d. The filesystem MCP server on one folder you don't mind it changing (Lesson 1.7).
   e. Personal memory in a gitignored folder, plus `example_memory/` with fake data.
   f. One subagent.
6. **Log failures as you go.** They feed your README's "what it can't do," your Canvas reflection, and Unit 2.

7. **Time-box every problem,** as the course asks. Past your limit, take it to Slack or class instead of another hour alone.

Vocabulary, in plain words

Each term is listed where it first shows up, so you can trace it back to its lesson.

Term	Plain meaning	First seen
API, API key	One program asking another to do something. The key says the request is yours; treat it like a password with a spending limit.	1.1
Model (LLM)	The part that reads text and writes text back. Here it's Gemini (`gemini-flash-lite-latest`).	1.1
LangChain	A Python library with one consistent way to call models and build agents that use tools.	1.1
Agent	A model plus a system prompt plus tools, which decides on its own whether to call a tool.	1.1
System prompt	Instructions that set who the agent is and how it behaves.	1.1
Tool	A Python function the agent can call. Two parts: the function (read by Python) and the docstring (read by the AI).	1.2
Docstring	The plain-English description under a function. It is the only thing the agent reads when choosing a tool.	1.2
`@tool`	A decorator that turns a function into a tool.	1.2
`.invoke()` / `.ainvoke()`	Run a tool or agent. `ainvoke` is the async version, required once MCP tools are involved.	1.2, 1.6
Community tools	Prebuilt tools such as DuckDuckGo search, Wikipedia, and Python REPL.	1.3
uv	An upgrade over pip that also creates and runs your project (uv init, uv add, uv run).	1.4

Term	Plain meaning	First seen
`.env`, `.gitignore`	The file that holds your secrets, and the list of files git must never commit.	1.4
Checkpointer, thread ID	Memory for a conversation, and the label for which conversation it is.	1.4
MCP (Model Context Protocol)	A standard way for agents to use tools that other people own and run. The slides call it USB for AI tools.	1.6
MCP server, MCP client	The server is the owner's program offering tools; the client (`MCPAdapter`) is the part of your agent that talks to it.	1.6
stdio vs HTTP	stdio: your agent starts the server on your machine (config says `command`). HTTP: it's already running elsewhere (config says `url`).	1.6, 1.7
`uvx`, `npx`	Download a Python (`uvx`) or Node (`npx`) program and launch it in one step.	1.7
Subagent	A second agent your main agent can hand work to, for example by using it as a tool.	Slide 1.3

Heads-up: the site is still being built

The home page says the course is in active development, so expect gaps; none of them mean you missed something.

- **Missing pages.** The lesson pages for 1.5, 1.6, and 1.8 return "not found." Use the slides for 1.5 to 1.7 instead, and Prep 1.7 is blank (the lesson says you need uv and Node.js).
- **Two schedules that disagree.** The Unit 1 page lists Day 4 as response formats, Day 5 as statefulness, and Day 8 as subagents. The home schedule shows what was actually taught (1.4 setup, 1.6 and 1.7 MCP). Treat the home schedule as the calendar and the Unit 1 page as the topic list.
- **Topics not yet on a published page:** response formats, statefulness, sandboxing (LangChain / Daytona), subagents, and writing prompts from scratch.

- **Two ways to build an agent in the code.** Lessons 1.2 and 1.3 use `create_react_agent(..., prompt=...)`; Lessons 1.1, 1.6, and 1.7 use `create_agent(..., system_prompt=...)`. The 1.6 slides warn this area changes fast: check the date on anything you copy, and compare an erroring snippet with the newer lessons.
- **Placeholders.** The 1.6 slides' clone link for the Summit Wireless repo and the 1.7 notebook link are marked as temporary in the slides themselves.
- **Units 2 and 3** have no pages yet. Their titles come from the course repository README.

Sources

Every page below was opened on Oct 9, 2026.

- Course home and schedule
- Unit 1: Build a "Bad" Agent
- Project 1: Personal Assistant Agent
- Lessons: 0 Get Oriented, 1.1, 1.2, 1.3, 1.4, 1.7
- Slides: 1.3 Group Day, 1.4, 1.5 Review, 1.6 Tools You Don't Own, 1.7 Someone Already Built It
- Preps: 1.1, 1.2, 1.3, 1.4, 1.6, 1.7
- Practice 1.2
- Course repository README (unit titles and weeks)